

Homework 1: Code Smells with Wahab and Johnny and Hector

So I looked through the movie rental program (Customer.java, Rental.java, Movie.java), and found a bunch of code smells — 20 to be exact. Some are small things, some are a bit bigger, but all of them could make the code messier or harder to maintain over time. Here's what I found:

In Customer.java

1. Long Method – statement()

This method is doing way too much. It calculates totals, builds the output string, and even handles formatting. It'd be cleaner if this stuff was split up into smaller methods.

2. Temporary Variables Everywhere

Variables like totalAmount and frequentRenterPoints only exist in the method and make things harder to read. They could be grouped or handled somewhere else to make the method less crowded.

3. Too Many Primitives

It uses plain ints for stuff like points. Would be better to wrap that in a class so it's easier to manage and change later.

4. Feature Envy

The method keeps asking Rental and Movie for their data to do all the work itself. It's like it's jealous — maybe the logic belongs over in those classes instead.

5. Repeated Code

The statement() and htmlStatement() methods do basically the same thing but in different formats. Instead of repeating everything, it could reuse shared logic.

6. Data Clumps

`totalAmount` and `frequentRenterPoints` always go together. That's a sign they should probably be part of their own class or result object.

7. Too Nosey (Inappropriate Intimacy)

The code reaches deep into objects like `rental.getMovie().getTitle()`. That's a lot of dot-chaining. It means `Customer` is too up in `Movie`'s business.

8. Vague Variable Name

`thisAmount` is super unclear. I had to read around it to realize it's the charge for the current rental. Just call it `rentalCharge` or something.

9. Switch Statement Smell

In general, switches based on type codes (like `getPriceCode()`) aren't ideal. It'd be better to use polymorphism, meaning each type could have its own class with its own behavior.

In Rental.java:

10. Middle Man

This file has getters that just pass things along, like `getTitle()`. Kinda pointless — it just forwards calls from another object without doing anything.

11. Feature Envy Again

The `getCharge()` method does a bunch of pricing logic based on what `Movie` it has. That logic really belongs to the `Movie`, not `Rental`.

12. Another Long Method

`getCharge()` is kind of bloated. It's all in one big switch block that could be cleaned up or broken apart.

13. Copy-Pasted Math

Each case in the switch block does its own little math snippet, but a lot of it looks the same. Probably could be extracted or reused better.

14. More Dot-Chaining (Too Intimate)

`getMovie().getPriceCode()` — again, reaching into another object's internals. It's tightly coupled and hard to change.

15. Weak Variable Naming

`daysRented` isn't awful, but something more descriptive like `rentalDurationInDays` could make things more clear.

In `Movie.java`

16. Magic Numbers (Type Code Smell)

This file uses static ints for categories like `CHILDRENS`, `REGULAR`, etc. Better to use enums or separate classes for each movie type.

17. Public Constants Everywhere

The type codes are public and global. That makes it too easy for other parts of the code to mess with them or misuse them.

18. Missed Opportunity for Polymorphism

Instead of using `switch(priceCode)`, each movie type could have its own class with its own pricing rules. That would make the logic easier to update.

19. Too Many Setters/Getters

It has plain old getter/setter methods with no logic. If `Movie` handled its own price behavior, you wouldn't need to expose all that data.

20. Lacks Real Responsibility

Movie holds the title and price code, but it doesn't do anything with them. It just stores stuff and lets other classes handle the logic, which isn't very object-oriented.